Name: ____Eric Hoffman___

MIDTERM

*Closed book except for one 8½ x 11 sheet. No calculators or electronics devices. All work is to be your own - **show your work** for maximum partial credit.*

1) (12pts) Multiple Choice

a. Two verilog assign statements drive a single signal of type wire. One drives with a weak strength and the other drives 1'bz. The resulting value of the signal will be:
   i. 1'bx
   ii. 1'bz
   iii. What ever value was being driven by the weak driver.
   iv. Will not compile because of the short circuit.

b. What can be said about the timing specified in the following verilog statement?

```
and #(1:2:3) (out_sig,a,b);
```

   i. Timing is specified for min, typical and max delays for **out_sig** rise.
   ii. Timing is specified for min, typical and max delays and are the same for rise/fall of **out_sig**
   iii. The synthesis tool will try to achieve these timings
   iv. Both I & iii
   v. Both ii & iii

Synthesis ignores timing. Took of 1 point instead of 1.5 if you answered i or v.

c. You work for Honda developing a steering sensor/power assist unit that will be common to the entire line of Honda cars. It monitors several redundant sensors that employ unique digital serial protocols. Choose the best technology.
   i. Full custom IC design
   ii. Standard cell based ASIC (Application Specific Integrated Circuit)
   iii. FPGA implementation (Field Programm
   iv. Micro-controller implementation.

Micro-controller not a bad answer. Only lost 0.5 for that. (unique protocols disqualifies it)

d. Analyze the code shown. What is the value of ***redor*** if ***full_bus*** = 8'hA5
   i. 1'b1
   ii. 1'b0
   iii. 8'h05
   iv. 8'hA5
   v. 1'bx

```
module bus_mask(input [7:0] full_bus,
                output redor);

  wire [7:0] mask_bus;

  assign mask_bus = 8'h0f & full_bus;
  assign redor = |mask_bus;

endmodule
```

e. For the code shown. What is the value of ***mask_bus***?
   i. 8'hA5
   ii. 8'h05
   iii. 1'b1
   iv. 4'h5
   v. 1'bx

f. To constrain timing of an output pin we need to inform Synopsys of:
   i. The drive strength of the gate driving the pin
   ii. The required timing of the pin relative to the ~~circuit input pins~~ clk
   iii. The capacitive load on the output.
   iv. All of the above

2) (7pts) Answer the questions for the code segment shown below:

```
 reg    out;     // type reg or wire?

always_comb begin
  casez({a,b})
    2'b01 : out = ~a;
    2'bz0 : out = b;
  endcase
end
```

(1.5pts) Fill in the type for signal **out**. You must choose from **reg** or **wire**.

(0.5pts) Is the begin/end pair for this always_comb block necessary?
(**Yes** / **No**) *circle one.*

(5pts) Using any desired mix of:

ANDs, ORs, NANDs, NORs, Latches, Flops, 2:1 Muxes, and Inverters draw how the **always_comb** block would synthesize.

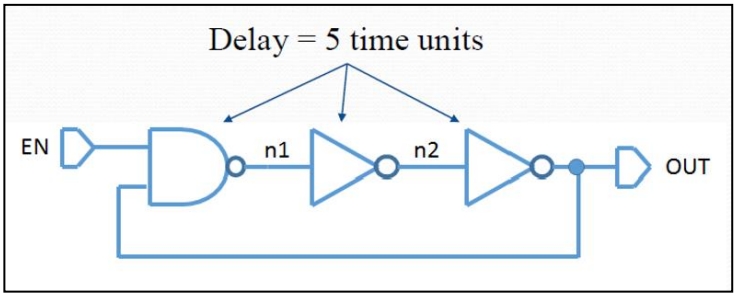
In the case of {a,b} = 11 it does not change, it remembers….a latch.

If you drew a combinational feed back loop you lost 2.5 points

If you drew a latch but had it wrong you lost 1.5 points.

If you drew logically correct but no sense of remembering in the 11 case you lost 3.5 points.

3) **(5pts)** The stretch goal of Ex01 was to implement the ring oscillator circuit shown in structural Verilog. **EN** is first held low, then later is raised. Since we have an odd number of inversions in a loop (ring) the circuit will start oscillating (15 time units high, 15 low). In the space provided code this using structural Verilog. *(just gate instances & connections not full module syntax)*



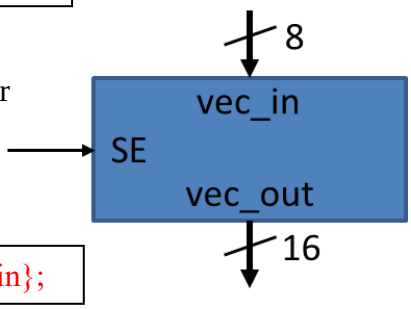
Most everyone had this correct, with the exception of specifying the delays. If you messed up the delays you got 1.5 points off.

4) **(5pts)** A behavioral implementation of the ring oscillator (see previous problem) is shown. Describe how it will simulate in ModleSim once **EN** is raised. If it would not simulate properly describe how you would change the code.

```
always @(EN,out)
  out = #15 (EN) ? ~out : 1'b1;
```

This evaluates the right hand side, then waits 15 time units and assigns it to out. However, it is still a blocking statement. Meanng it blocks until the assignment is made to out. Then the always block ends, and it never re-triggers. So it will not oscillate. To fix you would make it a non-blocking statement. There was a question a bit like this on the Cummings quiz (the clock generator question). Many lost 3 to 3.5 point on this.

5) **(4pts)** Write a dataflow (***assign***) statement to either zero extend or sign extend an incoming 8-bit signal (**vec8**) to form outgoing 16-bit signal (**vec16**). If the control signal **SE** is high then it should sign extend.



assign vec_out = (SE) ? {{8{vec_in[7]}},vec_in} : {8'h00,vec_in};

Most people got this one. Some lost 0.5pts for missing a set of { }

6) **(3pts)** Verilog & VHDL are the two dominant HDLs. VHDL only has non-blocking statements and does not have anything equivalent to the **assign** statement, so all combinational has to be inferred in something akin to an always block. Fill in the sensitivity list for the combinational always block shown.

```
always@( a,b,c,en,stg1 ) begin
    stg1 <= en? a^b : a|c;
    out1 <= stg1 & c;
end
```

7) **(4pts)** Given the **initial** blocks shown (that are part of a test bench). What are the values of **a** and **b** at time 5?

```
initial begin
    a = 0;
    b = 1;
end

initial begin
    a = 1;
    #5 b = 0;
end
```

a is 0 or 1 but we don't know (verilog race condition)
b = 0;

people did pretty good on this one

8) **(5pts)** Shown is partial code for the **always_comb** block of a SM. The ellipses (…) represent code that is not shown. We don't know the exact desired functionality of this SM, however, we can still point out an obvious mistake. Explain **how** the error would manifest itself.

```
always_comb begin
  ...          ;// defaults
  case (state)
    IDLE: begin
      ...      ;// some stuff;
    end
    TX : begin
      if (shft) begin
        bit_cnt = bit_cnt + 1;
        nxt_state = SMPL;
      end else
        nxt_state = TX;
    end
    ...        ;//more cases
  endcase
end
```

bit_cnt = bit_cnt + 1;

How can that be inside an always_comb block?

Clearly a combinational feedback loop.

Perhaps a 1/3 of students got this right. Many lost 3.5pts on this one.

9) **(5pts)** Draw how this code would synthesize.

```
always @(posedge clk)
    if (!rst_n)
      q1 <= 1'b0;
    else begin
      q1 <= d;
      q2 <= q1;
    end
```

I covered this example twice, and once was during the midterm review session the day of the test. Most of you were in the room. Still only about ¼ of the class drew this right.

Arrrgh!

In the "if (!rst_n)" condition q2 <= q2 is inferred so there is actually a 2:1 mux between q1 & q2

10) **(5pts)** Which of the following parameter types available in Verilog would be the most appropriate for use where you are parameterizing a value and possibly overriding that default value when instantiating the module.

Circle one:

a) localparam
b) defparam
c) parameter
d) #define

Briefly explain why the other choices are not a good choice

localparam can't be overridden
There was another Cumming's paper posted called "Death to defparam. But since I didn't quiz you on it you probably didn't read it.
#define is global and also cannot be overridden.
Plus it is 'define not #define. #define is C.

Most got this one right. We were forgiving on your reason for defparam.

11) (6pts) We are looking at a stream of bits (eg. 0101001100110…) where bits arrive one per clock cycle. Our module looks at a sliding window of 8 bits over this stream and asserts a ***match*** signal for any clock period in which the most recent 8-bits match 8'hAB *(assume MSB is oldest bit of pattern)*. Complete the code. *(can't believe I am saying this…but **don't** think statemachine for this)*

```
module pattern_match(input clk, rst_n, in_stream, output match);

reg [7:0] shft_reg;

always_ff @(posedge clk, negedge rst_n)
  if (!rst_n)
    shft_reg <= 8'h00;
  else
    shft_reg <= {shft_reg[6:0],in_stream};

assign match = (shft_reg == 8'hAB) ? 1'b1 : 1'b0;

endmodule
```

12) (4pts) Answer the questions below about the code snippet implementing a rising edge detector.

```
reg ff1,ff2,ff3,rising;

always @(posedge clk) begin
  ff3 = ff2;
  ff2 = ff1;
  ff1 = sig_in;
end

assign rising = ff2 & ~ff3;
```

(1.5pts) True/False: The input ***sig_in*** must have been asynchronous to the clock domain clk.

Why double flop if it wasn't? Only took of 1 point

(2.5pts) The code is incorrect because (circle one):

a.) The logic is that of falling edge, not rising edge detection.
b.) ***rising*** should have been declared as type **wire** not **reg**
c.) Flops of an edge detector should be reset, and these are not
d.) Non-Blocking statements should have been used for ff1,ff2,ff3.
e.) All of the above
f.) **b, c, d** are all true
g.) **b** & **d** are true

Only took off 1pt for g, but edge detector flops should be reset or preset. Remember our PB switch flops had to be preset.

13) (4pts) Explain when you would use **$stop** vs **$finish** in a testbench.

Most did pretty good on this. Still you could have lost a point if you didn't make it clear that $finish would be used in a scenario where you were running tests in batch mode from some form of script or automation.

14) (6pts) (4pts of 6pts) What is this module implementing?

```
module mystery_module(input clk,rst_n; input [7:0
                      output intensity);

  always @(posedge clk, negedge rst_n)
    if (!rst_n)
      cntr <= 8'h00;
    else
      cntr <= cntr+1;

  always @(posedge clk, negedge rst_n)
    if (!rst_n | clear)
      intensity <= 1'b0;
    else if (set)
      intensity <= 1'b1;

  assign set = ~|cntr;          // set when cntr==0x00
  assign clear = (cntr>=mag) ? 1'b1 : 1'b0;

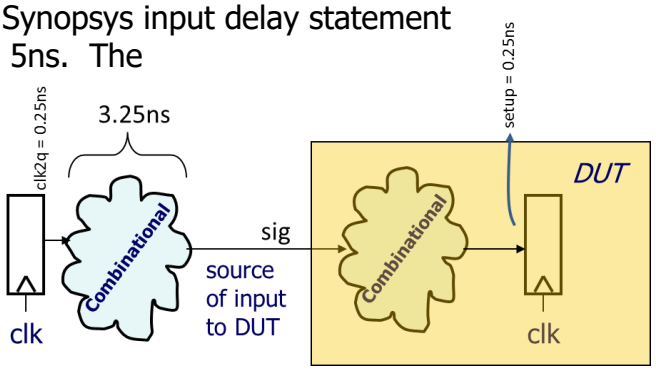
endmodule
```

Many gave me a decent description but still lost 1.5 points. It is a PWM. You made one already. You should be able to recognize it.

(2pts) The **always** block for implementing ***intensity***:

a) Is just fine as is
b) Will infer a latch because there is no **else** to the **if**.
c) Will not synthesize correctly because of the: (!rst_n | clear) in the reset clause of the **if**.
d) **Needs** to be separated into a simpler always block and some form of combinational logic

15) (4pts) For the circuit shown below finish the Synopsys input delay statement for the signal ***sig*** into your DUT. The clock period is 5ns. The flops in your design have a clk2q delay of 0.25ns and a setup time of 0.25ns.?



create_clock –name "clk" –period 5 clk

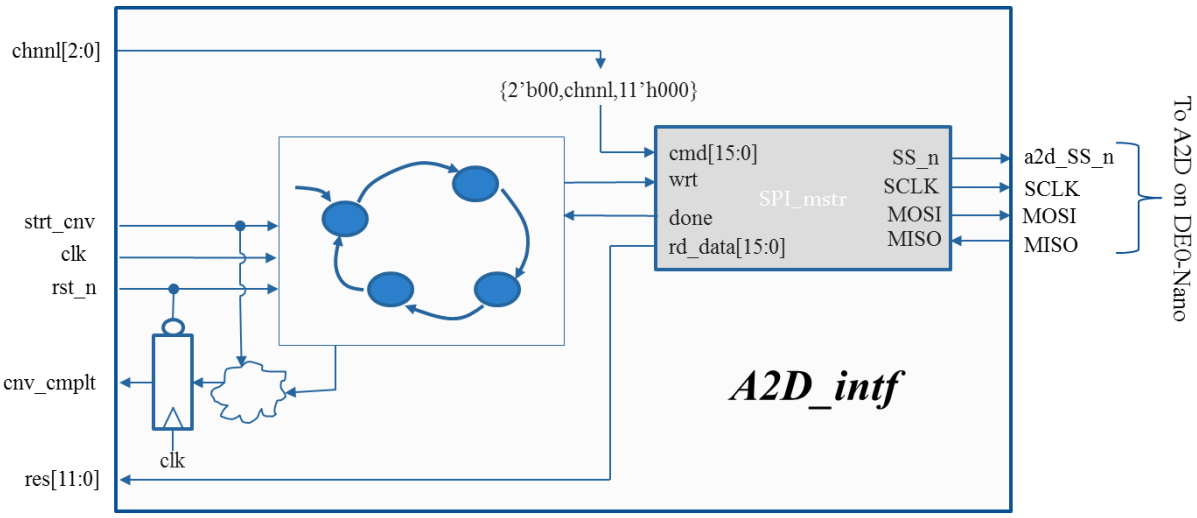
set_input_delay 3.5 -clock clk sig

Didn't have to have the syntax perfect to get full credit. If you had the 3.5 and something close.

16) (18 pts) In HW3 you created a SPI master. We are now going to use that SPI master to create an interface to the A2D on the DE0-Nano board. The basic steps are:

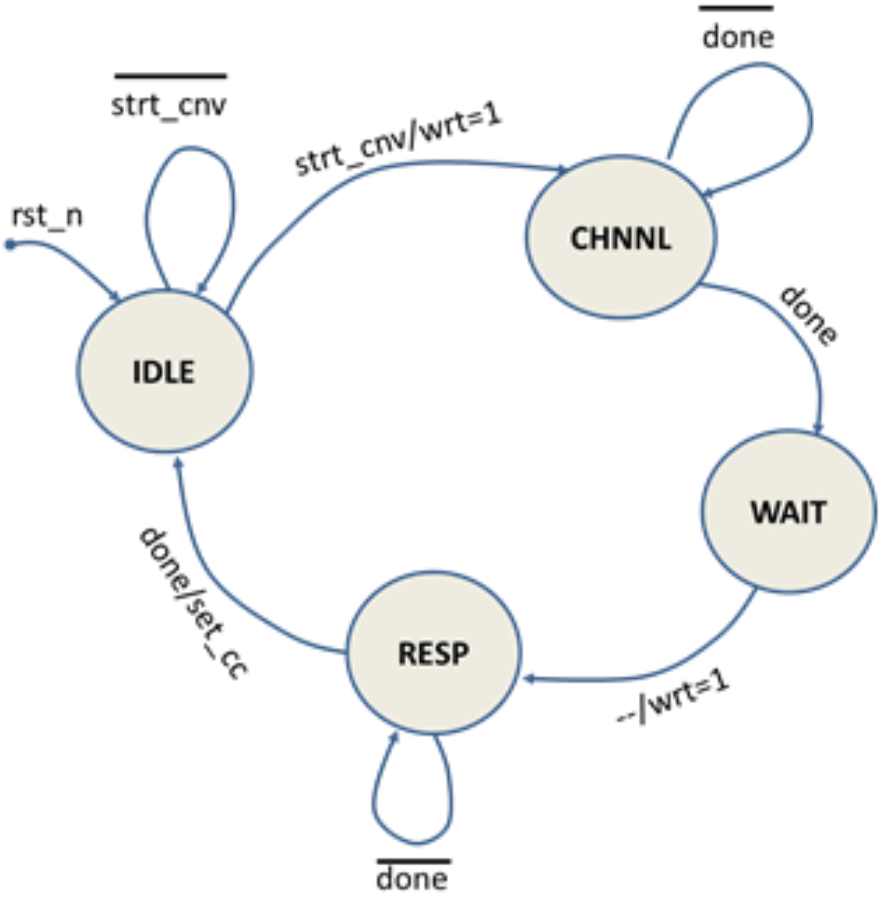
When ***strt_cnv*** is asserted initiate a SPI transaction to inform the A2D what channel to convert (A2D returns garbage during this transaction).
Wait one clock cycle after the SPI transaction has completed.
Initiate a SPI transaction to grab the data returned from the A2D (not so important what command was sent to A2D during this transaction).
Assert ***conv_cmplt*** when A2D data is ready on ***res[11:0]*** (result)

<<See diagram next page>>



SM Function: Send command to A2D via SPI to ask for conversion on channel. Once that transaction completes wait one clock cycle. Then start new transaction to read the result of the A2D conversion back.

Draw a bubble diagram for the state machine:



Finish the code to implement A2D_intf including SM functionality.

```
module A2D_intf(input clk,rst_n,strt_cnv; input [2:0] chnnl; output reg cnv_cmplt;
                output [11:0] res; output a2d_SS_n,SCLK,MOSI input MISO);
  wire [15:0] cmd;
  wire done;
  ///// define outputs of SM as type logic /////
  logic wrt, set_cc;
  reg cnv_cmplt;
  ///// defines for states for SM below //////
  typedef enum reg[1:0] {IDLE,CHNNL,WAIT,RESP} state_t;
  state_t state, nxt_state;
  ///// Instantiate SPI master for A2D interface /////
  SPI_mstr16 iSPI(.clk(clk),.rst_n(rst_n),.SS_n(a2d_SS_n),.SCLK(SCLK),.MISO(MISO),
                  .MOSI(MOSI),.wrt(wrt),.done(done),.rd_data(res),.cmd(cmd));
   always_ff @(posedge clk,negedge rst_n)
     if (!rst_n)
       state <= state;
    else
       state <= nxt_state;
   always_ff @(posedge clk, negedge rst_n)
      if (!rst_n)
        cnv_cmplt <= 1'b0;
     else if (strt_cnv)
        cnv_cmplt <= 1'b0;
     else if (set_cc)
       cnv_cmpl <= 1'b1;
   assign cmd = {2'b00,chnnl,11'b000};
   always_comb begin
     wrt = 0;
     set_cc = 0;
     nxt_state <= state;
     case (state)
       IDLE : if (strt_cnv) begin
         wrt = 1;
         nxt_state = CHNNL;
       end
       CHNNL : if (done)
         nxt_state = WAIT;
       WAIT : begin
         nxt_state = RESP;
         wrt = 1;
       end
       RESP : if (done) begin
         nxt_state = IDLE;
         set_cc = 1;
       end
     endcase
   end
endmodule
```